

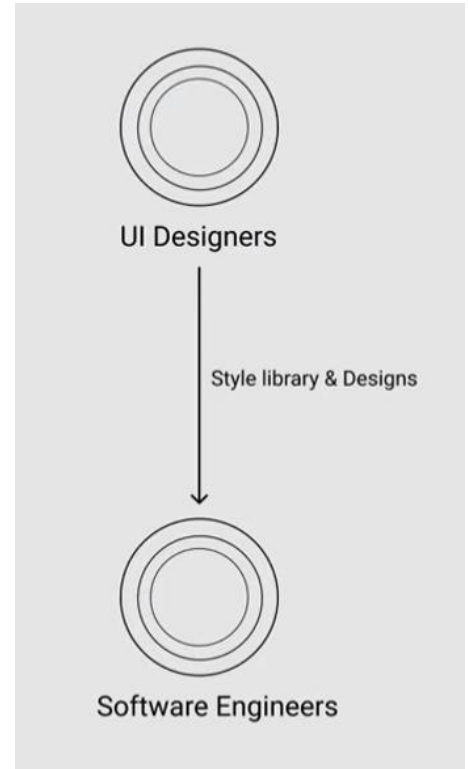
Fundamentals of Web Development

Third Edition by Randy Connolly and Ricardo Hoar

Chapter 3

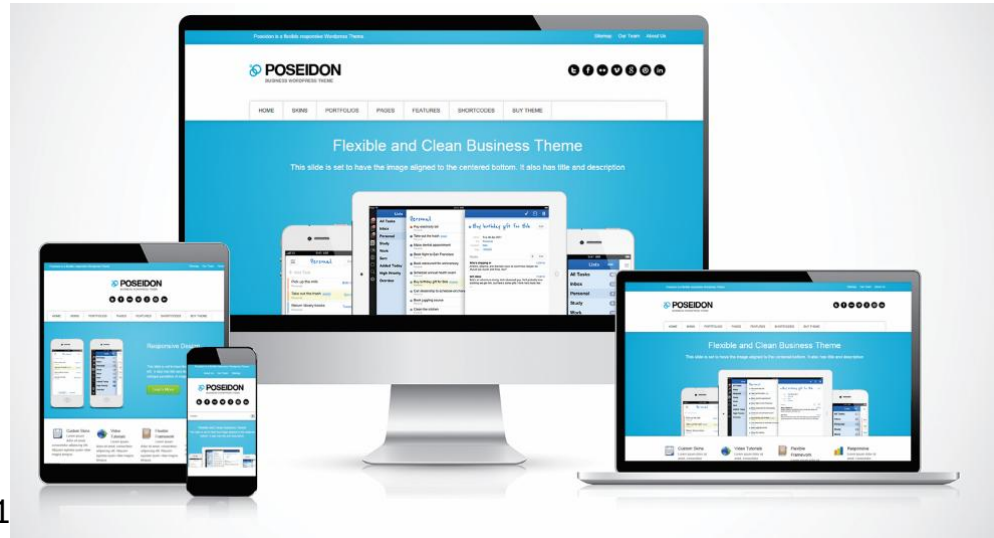
(In the book this is chapters 4 and 5)

CSS part 1: Selectors and
Basic Styling



What Is CSS?

- CSS is a W3C standard for describing the **appearance** and the **layout** of an HTML document.
- With CSS, we can assign font properties, colors, sizes, borders, background images, positioning and even animate elements on the page.
- CSS allows Responsive Design

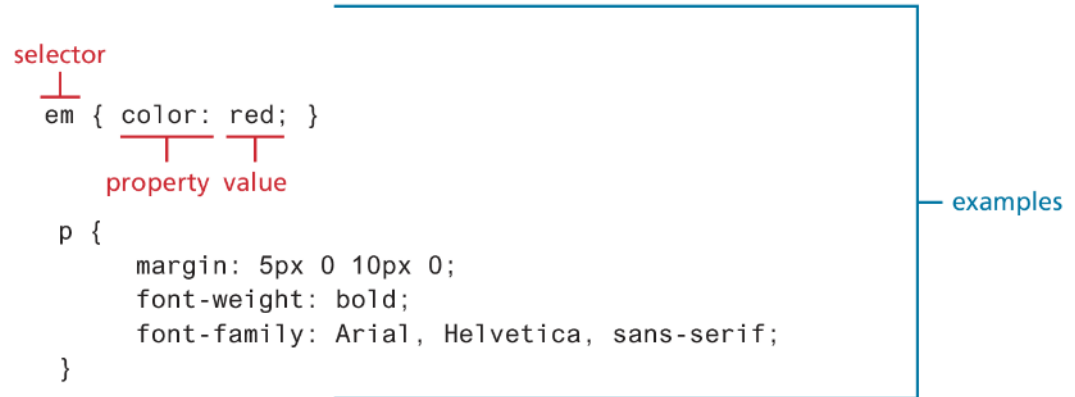
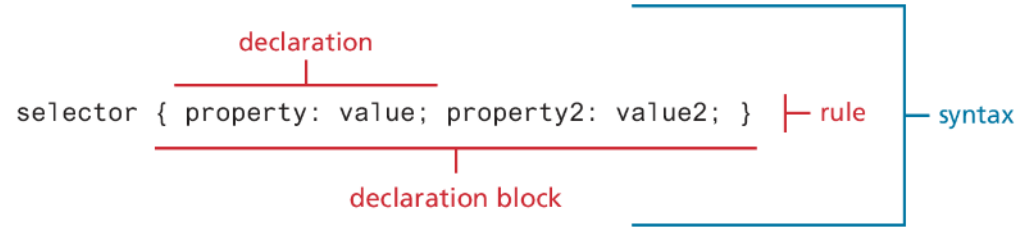


Benefits of CSS

- **Main engineering concepts: Separation of concerns + Reuse**
- **Improved control over formatting.**
- **Improved site maintainability.** All formatting can be centralized into one CSS file
- **Improved accessibility.** By keeping presentation out of the HTML, screen readers and other accessibility tools work better.
- **Improved page-download speed.** Each individual HTML file will contain less style information and markup, and thus be smaller.
- **Improved output flexibility.** CSS can be used to adapt a page for different output media. This approach to CSS page design is often referred to as **responsive design**.

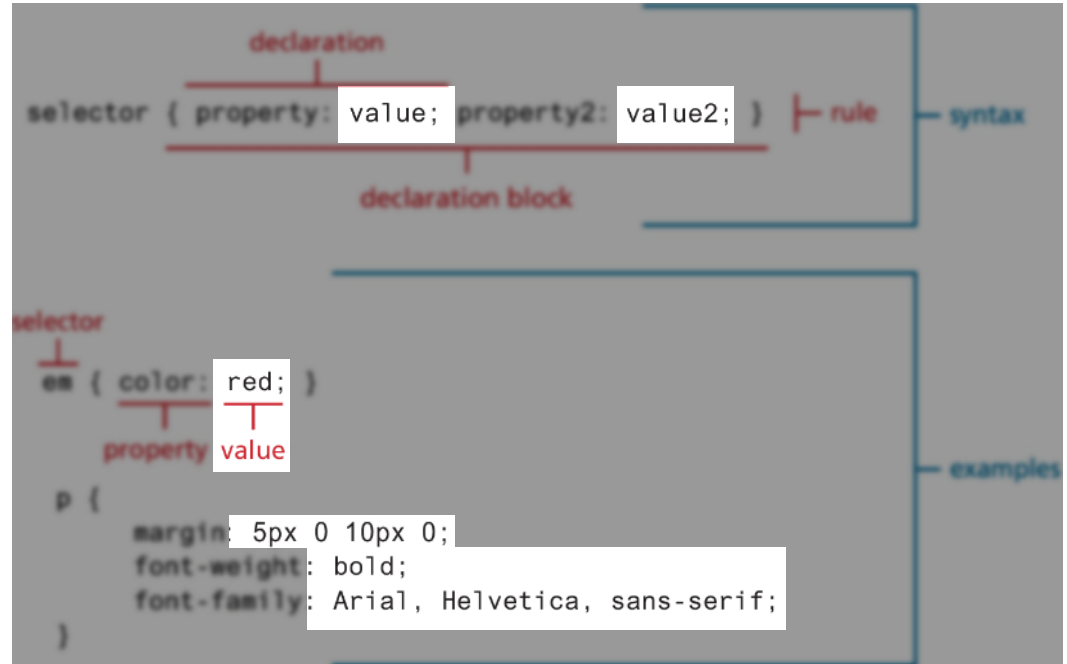
CSS Syntax

- A CSS document consists of one or more **style rules**.
- A rule consists of a **selector** that identifies the HTML element or elements that will be affected, followed by a series of **property:value pairs** called **the declaration**
- The series of declarations is also called the **declaration block**.



CSS Syntax: Values

- Each individual CSS declaration must also contain a **value**
- The unit of any given value is dependent upon the property.
- Some property values are from a predefined list of keywords. Others are, percentages, values such as length measurements numbers without units, color values, and URLs.



Color Values

Method	Description	Example
Name	Use one of 17 standard color names. CSS3 has 140 standard names.	color: red; color: hotpink; /* CSS3 only */
RGB	Uses three different numbers between 0 and 255 to describe the red, green, and blue values of the color.	color: rgb(255,0,0); color: rgb(255,105,180);
Hexadecimal	Uses a six-digit hexadecimal number to describe the red, green, and blue value of the color	color: #FF0000; color: #FF69B4;
RGBA	This defines a partially transparent background color. The “a” stands for “alpha,” which is a term used to identify a transparency	color: rgba(255,0,0,0.5); (check: rgba-colorpicker.com)
HSL	Allows you to specify a color using Hue Saturation and Light values. This is available only in CSS3. HSLA is also available as well.	color: hsl(0,100%,100%); color: hsla(330,59%,100%,0.5);

Common Units of Measure Values

Units of measure in CSS are either

- **relative units**, in that they are based on the value of something else, or
- **absolute units**, in that they have a real-world size.

Some example measures:

- **px** Pixel. In CSS2 this is a relative measure, while in CSS3 it is absolute (1/96 of an inch -> 1px ~ 1.06mm)
- **em** Equal to the computed value of the font-size property of the element on which it is used (2em means 2 times the size of the current font)
- **vw** Relative to 1% of the width of the viewport (the browser window size). If the viewport is 30cm wide, 1vw = 0.3cm.
- **in** Inches (absolute)
- **cm** Centimeters (absolute)

Location of Styles

CSS style rules can be located in three different locations.

- Inline Styles
- Embedded Style Sheet
- External Style Sheet

These three **are not mutually exclusive**, in that you could place your style rules in all three.

Inline Styles

Inline styles are style rules placed within an HTML element via the style attribute

- An inline style only affects the element it is defined within and overrides any other style definitions for properties used

```
<h1>Share Your Travels</h1>  
<h2 style="font-size: 24pt">Description</h2>  
...  
<h2 style="font-size: 24pt; font-weight: bold;">Reviews</h2>
```

LISTING 4.1 Inline styles example

Embedded Style Sheet

Embedded style sheets (also called internal styles) are style rules placed within the `<style>` element (inside the `<head>` element of an HTML document)

- While better than inline styles, using embedded styles is also discouraged.

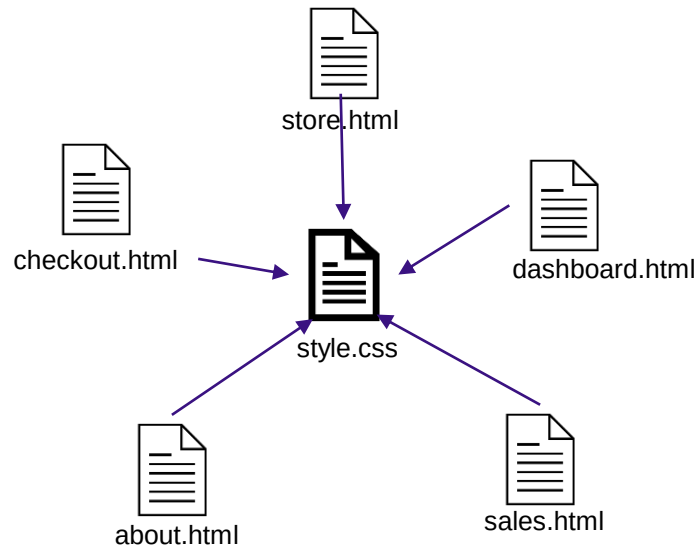
```
<head>
  <meta charset="utf-8">
  <title>Chapter 4</title>
  <style>
    h1 { font-size: 24pt; }
    h2 {
      font-size: 18pt;
      font-weight: bold;
    }
  </style>
</head>
<body>
```

LISTING 4.2 Embedded styles example

External Style Sheet

External style sheets are style rules placed within an external text file with the **.css** extension.

- When you change an external style sheet, all HTML documents that reference that style sheet will **automatically use the updated version**.
- The browser is able to **cache** the external style sheet, which can improve performance as well.



```
<head>
  <meta charset="utf-8">
  <title>Chapter 4</title>
  <link rel="stylesheet" href="styles.css" />
</head>
```

LISTING 4.3 Referencing an external style sheet

Source code: [chapter03/00-inline-external-embed](#)

Selectors

When defining CSS rules, you will need to use a selector.

- Selectors tell the browser which elements will be affected by the property values
- Selectors allow you to select individual or multiple HTML elements.
- Three basic selector types have been around since the earliest CSS2 specification.
 - Element Selectors
 - Class Selectors
 - Id Selectors

Element Selectors

Element selectors select all instances of a given HTML element.

You can also select all elements by using the **universal element selector** (*)

You can select a group of elements by separating the different element names with commas.

```
/* commas allow you to group selectors */
p, div, aside {
    margin: 0;
    padding: 0;
}
/* the above single grouped selector is equivalent to the
following: */
p {
    margin: 0;
    padding: 0;
}
div {
    margin: 0;
    padding: 0;
}
aside {
    margin: 0;
    padding: 0;
}
```

LISTING 4.4 Sample grouped selector

Class and ID Selectors

A **class selector** allows you to target different HTML elements labeled with the same class

We use a period (.) followed by the class name.

An **ID selector** allows you to target a specific element by its id attribute, which takes the form:

hash (#) followed by the id name.



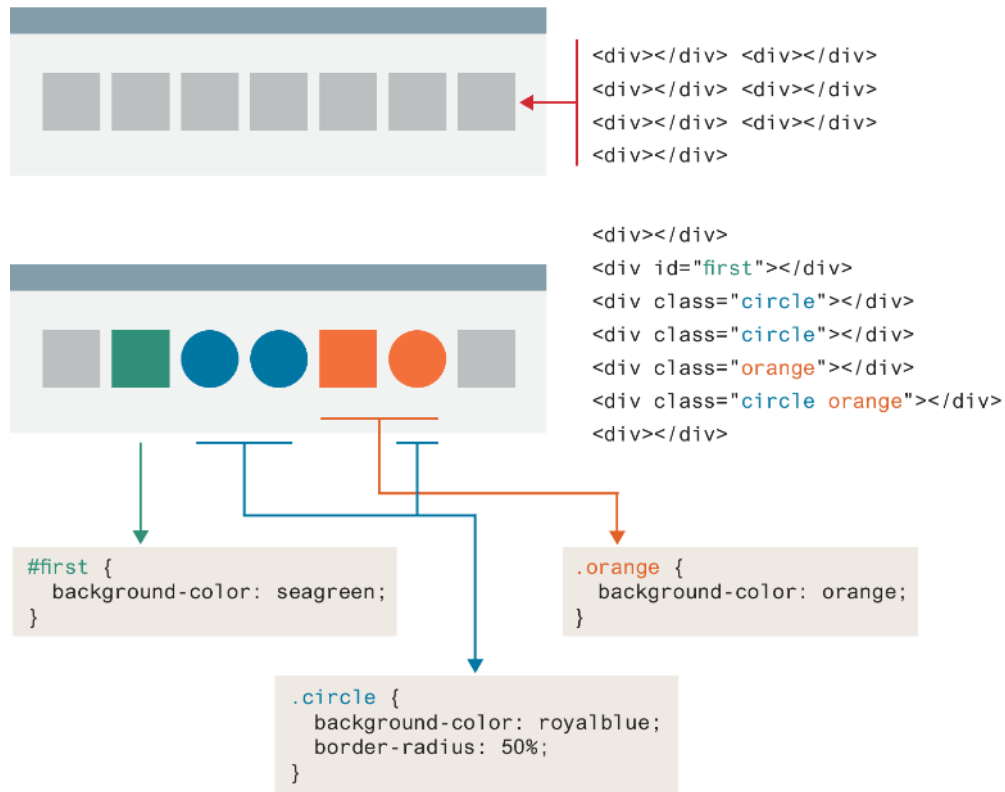
The difference between “class” and “id”:

“id” is **unique** in a document and used to target only one element, whereas “class” can be used to target many elements.

Class and ID Selector Example

Consider this figure, the original HTML can be modified to add class and id values, which are then styled in CSS.

- Id Class select selector **#first** matches the div with id “first”
- Class selectors **.orange** and **.circle**, match all elements with those class values.
- Notice that an element can be tagged with multiple classes (**priority** goes to the last rule in the document).



Note: in reality the divs would appear vertically

Source code: [chapter03/01_selectors](#)

Attribute Selectors

An **attribute selector** provides a way to select HTML elements either by the presence of an element attribute or by the value of an attribute.

- Attribute selectors can be helpful in the styling of hyperlinks and form elements
- These selectors can be combined with the element id and class selectors.
- You **use square brackets** to create attribute selectors

Attribute Selectors

	Matches	Example
[]	A specific attribute.	[title] Matches any element with a title attribute
[=]	A specific attribute with a specific value.	a[title="posts from this country"] Matches any <a> element whose title attribute is exactly "posts from this country"
[~=]	A specific attribute whose value matches at least one of the words in a space-delimited list of words.	[title~="Countries City"] Matches any title attribute that contains the word "Countries" w3schoolsExample
[^=]	A specific attribute whose value begins with a specified value.	a[href^="mailto"] Matches any <a> element whose href attribute begins with "mailto"
[*=]	A specific attribute whose value contains a substring.	img[src*="flag"] Matches any element whose src attribute contains somewhere within it the text "flag"
[\$=]	A specific attribute whose value ends with a specified value.	a[href\$=".pdf"] Matches any <a> element whose href attribute ends with the text ".pdf"

Source code: [chapter03/01_selectors](#)

Attribute Selector Example

Here we show an attribute selector to show PDF files differently than other links

```
a[href$=".pdf"] {  
  background: url(pdf_icon.svg) no-repeat left center;  
  padding-left: 19px;  
}
```

This attribute selector selects only those `<a>` elements whose href attribute value ends with the characters `".pdf"`.

```
<a href="abc.html">First link</a>  
<a href="sample.pdf">One PDF</a>  
<a href="another.pdf">Two PDF</a>
```

First link  One PDF  Two PDF

Source code: [chapter03/01_selectors](#)

Pseudo-Element and Pseudo-Class Selectors

A **pseudo-element selector** is a way to select something that does not exist explicitly **as an element** in the HTML document tree.

- You can select the **first line** or **first letter** of any HTML element using a pseudo-element selector.

A **pseudo-class selector** targets either a **particular state** or a **variety of family relationships**

- This type of selector is for targeting link states and for adding hover styling for other elements

Common Pseudo-Class and Pseudo-Element Selectors

- **a:link** Selects links that have not been visited.
- **a:visited** Selects links that have been visited.
- **:hover** Selects elements that the mouse pointer is currently above.
- **:active** Selects an element that is being activated by the user. A typical example is a link that is being clicked.
- **:first-child** Selects an element that is the first child of its parent.
- **:first-letter** Selects the first letter of an element.
- **:first-line** Selects the first line of an element.
- **:is()** takes a selector list and selects any element that can be selected by any of the selectors in that list (example **`:is(input.label.button.select):hover`** instead of **`input:hover, label:hover, ...`**)

Styling a link using pseudo-class selectors

Home Mens Womens Kids **House** Garden Contact



```
li:hover { ... }
```

Home **Mens** Womens Kids House Garden Contact

```
li:first-child { ... }
```

Home Mens **Womens** Kids House Garden **Contact**

```
li:nth-child(2n) { ... }
```

Home Mens Womens Kids **House** **Garden** **Contact**

```
li:nth-child(2n-1) { ... }
```

[Arsenal](#)

[Chelsea](#)

[Liverpool](#)



[Manchester United](#)



click

[West Ham United](#)

```
a:link { ... }
```

```
a:visited { color: royalblue }
```

```
a:hover { color: lavender; background-color: hotpink }
```

```
a:active { font-weight: bold }
```

```
a:link:last-child { text-decoration: none }
```

Pseudo selectors can be combined

Syntax of a descendant selection

context selected element

div p { ... }

A red line starts from the 'div' context, goes down, then right, then up to the 'p' selected element.

Selects a <p> element
somewhere
within a <div> element

#main div p:first-child { ... }

A red line starts from the '#main' context, goes down, then right, then up to the 'p' selected element.

Selects the first <p> element
somewhere within a <div> element
that is somewhere within an element
with an id="main"

Task

- Style (color: red) last child of the unordered list
- Style <a> where href has substring “example”

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  |   <title>CSS Task</title>
5  |   <style>
6  |     |   ul li:last-child {
7  |       |   color: red; }
8  |       |   a[href*="example"] {
9  |         |   color: red;
10 |       |   }
11 |     </style>
12 </head>
13 <body>
14 |   <header>
15 |     |   <h1>Website B</h1>
16 |     </header>
17 |   <main>
18 |     |   <ul>
19 |       |   <li>Item 1</li>
20 |       |   <li>Item 2</li>
21 |       |   <li>Item 3</li>
22 |     </ul>
23 |     <a href="https://www.example.com">Link 1</a>
24 |     <a href="https://www.test.com">Link 2</a>
25 |     <a href="https://www.example.org">Link 3</a>
26 |     <a href="https://www.other.com">Link 4</a>
27 |   </main>
28 |   <footer>
29 |     |   <hr>
30 |     |   <p>This is the footer of the page.</p>
31 |   </footer>
32 </body>
33 </html>
```

← → ↻ ⓘ 127.0.0.1:56330/task/task.html

Website B

- Item 1
- Item 2
- Item 3

[Link 1](#) [Link 2](#) [Link 3](#) [Link 4](#)

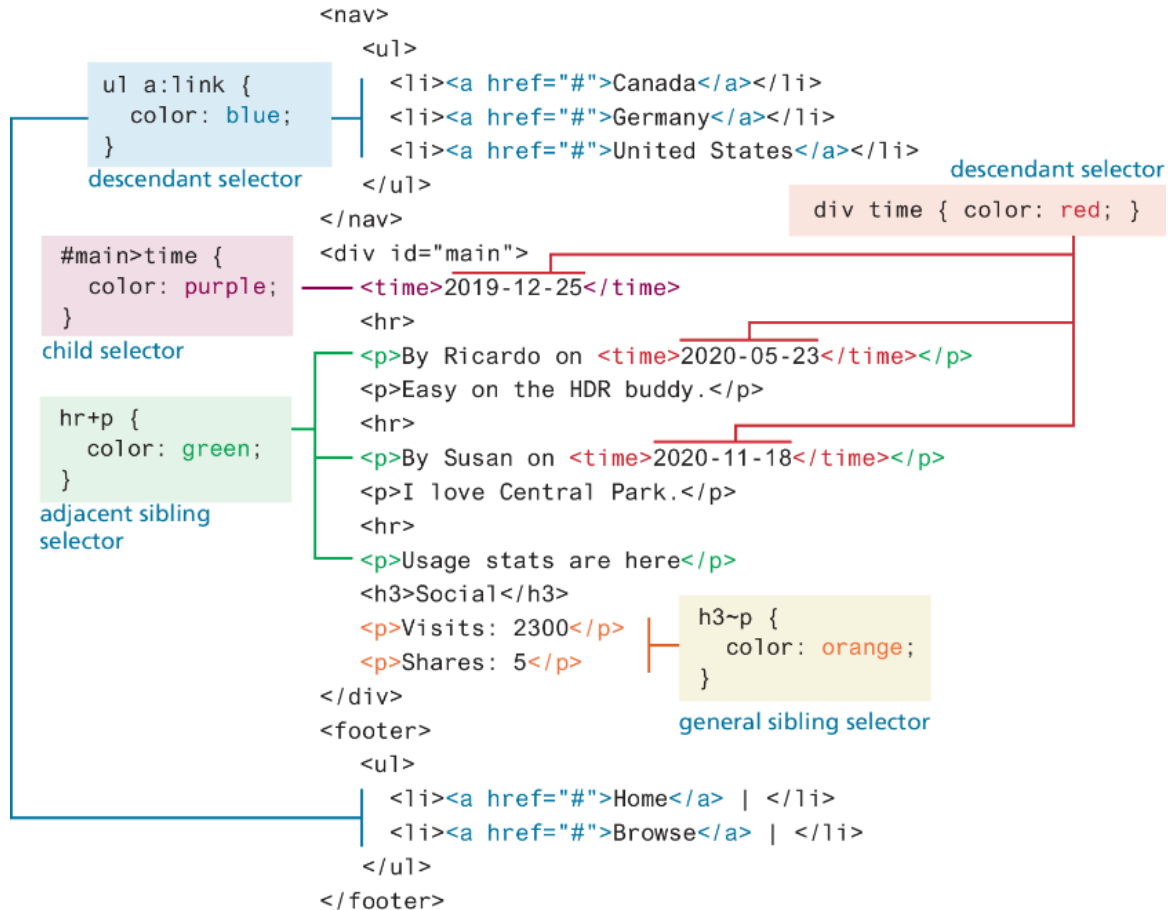
This is the footer of the page.

Contextual Selectors

A **contextual selector** (in CSS3 also called **combinators**) allows you to select elements based on their *ancestors*, *descendants*, or *siblings*.

- A **descendant selector** matches all elements that are contained within another element. The character used to indicate **descendant** selection is a space " "
- A **child selector** matches a specified element that is a **direct child** of the specified element. The character used is a ">"
- An **Adjacent selector** matches a specified element that is the **next sibling** of the specified element. The character used is a "+"
- A **General sibling selector** matches All **following** elements that shares the same parent as the specified element. The character used is a "~"

Contextual selectors in action



NEW! Nested CSS rules

Ref: <https://drafts.csswg.org/css-nesting-1/>

- Still a work in progress by W3C, but implemented by all major browsers (see: <https://caniuse.com/css-nesting>)
- It is a simpler way of writing CSS rules.

```
parent {  
  /* parent styles */  
  & child1 {  
    /* child1 styles */  
  }  
  & child2 {  
    /* child2 styles */  
  }  
}
```

=

```
parent {  
  /* parent styles */  
}  
parent child1 {  
  /* child styles */  
}  
parent child2 {  
  /* child styles */  
}
```

The Cascade: How Styles Interact

The “Cascade” in CSS refers to how conflicting rules are handled.

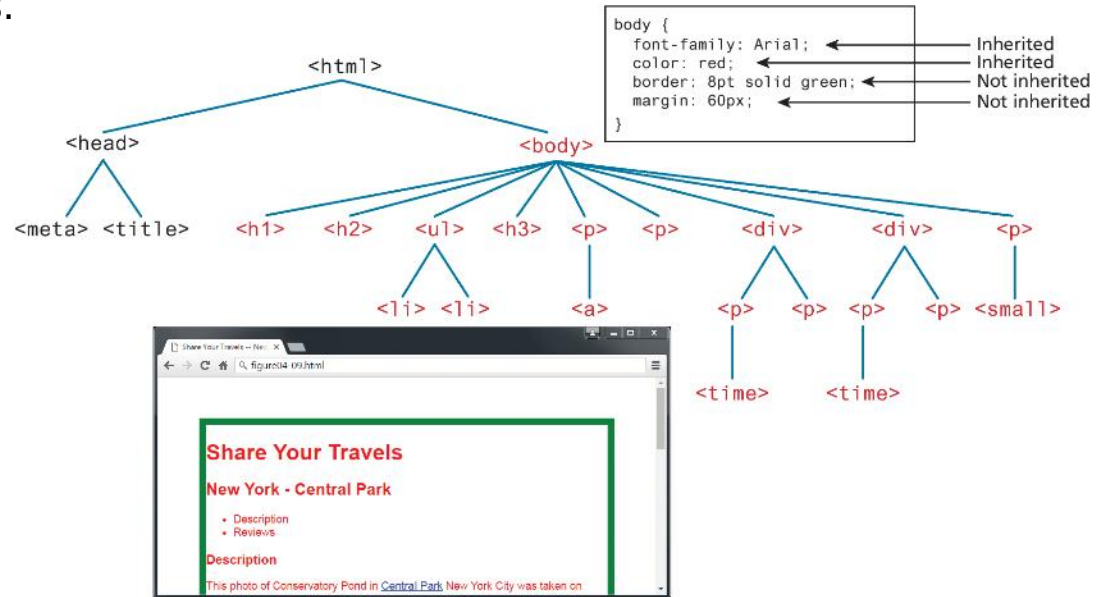
CSS uses the following cascade principles to help it deal with conflicts:

- inheritance,
- specificity, and
- location.

The Cascade: Inheritance

Inheritance is the principle that many CSS properties affect their descendants as well as themselves.

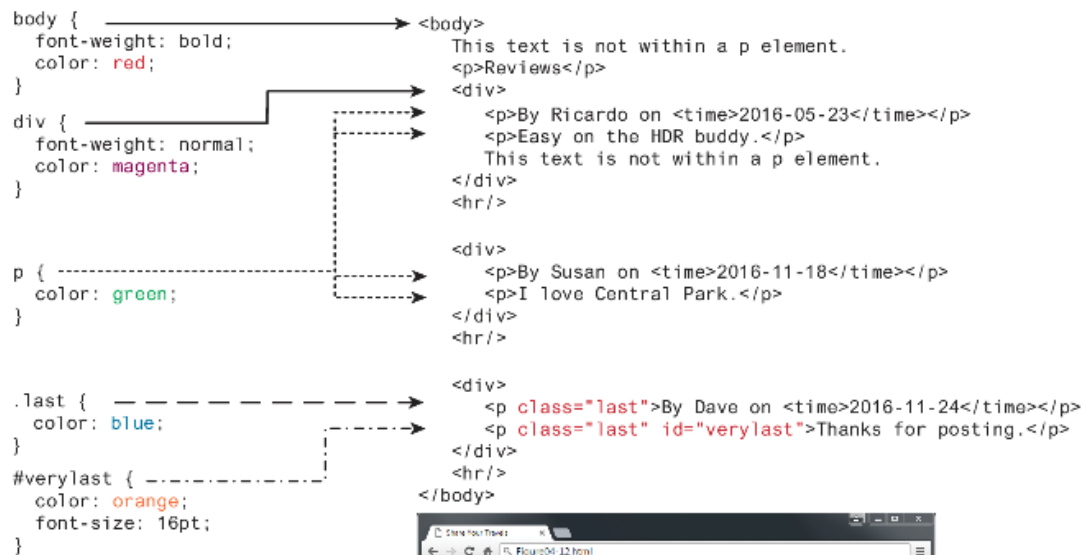
- Font, color, list, and text properties are inheritable;
- Layout, sizing, border, background, and spacing properties are not.
- it is also possible to inherit properties that are normally not inheritable using **inherit**
{color: inherit;}



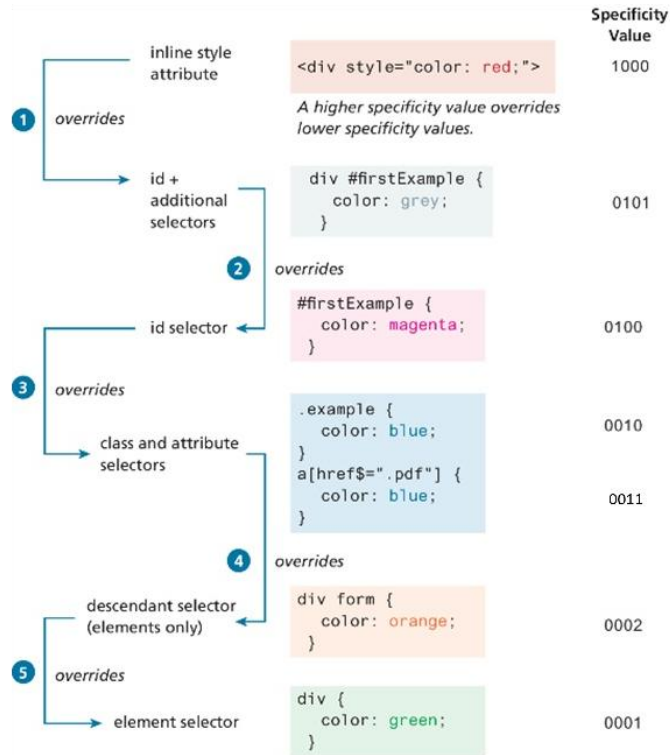
The Cascade: Specificity

Specificity is how the browser determines which style rule takes precedence when more than one style rule could be applied.

- The **more specific selector** takes precedence
- **Class selectors take precedence over element selectors, and id selectors take precedence over class selectors**
- *Note: `<body>` color and font-weight properties are overridden by the more specific `<div>` and `<p>` selectors.*



Simplified Specificity algorithm



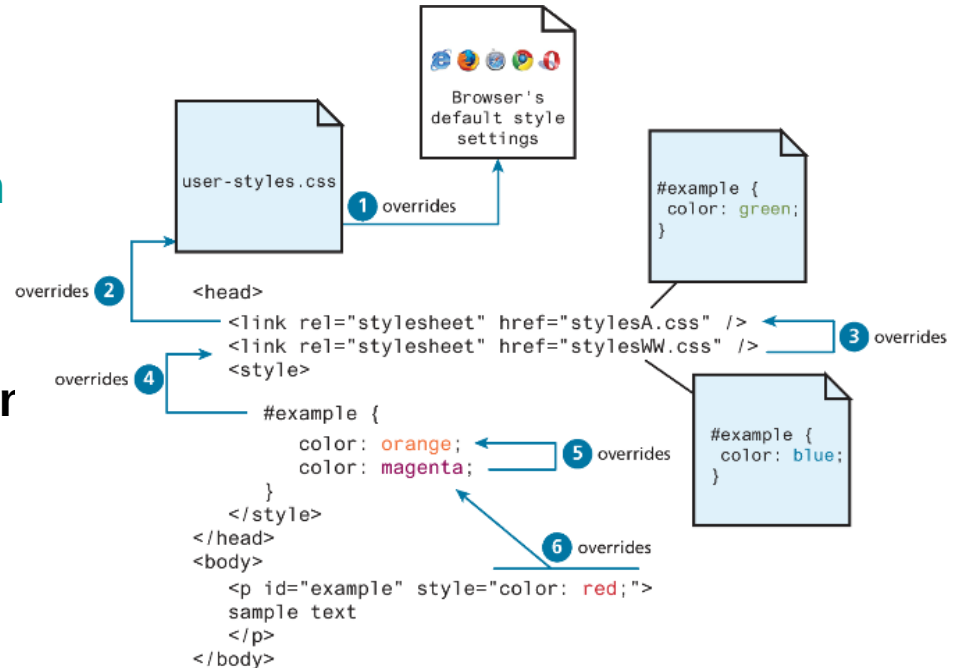
a= inline style
b= id
c= count attribute/class
d= count element
abcd

The Cascade: Location

Finally, when inheritance and specificity cannot determine style precedence, the principle of **location** will be used.

When rules have the same specificity, then the latest are given more weight.

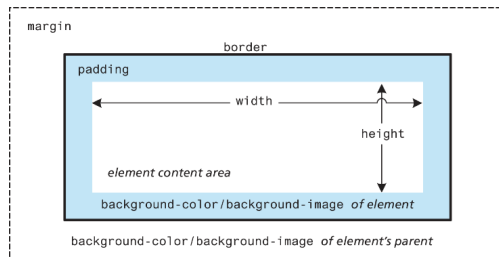
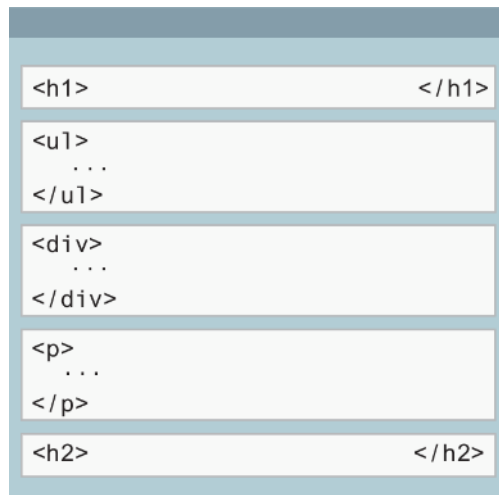
For instance, an inline style will override one defined in an external author style sheet or an embedded style sheet.



Block Elements

Block-level elements such as `<p>`, `<div>`, `<h2>`, `<ul>`, and `<table>` are each contained on their own line.

- without styling, two block-level elements can't exist on the same line
- **Block-level elements use the normal CSS box model**



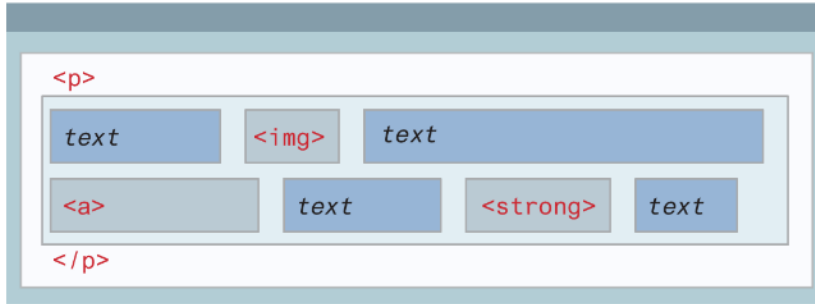
Inline Elements

Inline elements do not form their own blocks but instead are displayed within lines.

- Normal text in an HTML document is inline, as are elements such as `<em>`, `<a>`, `<img>`, and `<span>`.
- When there isn't enough space left on the line, the content moves to a new line

Inline Element Example

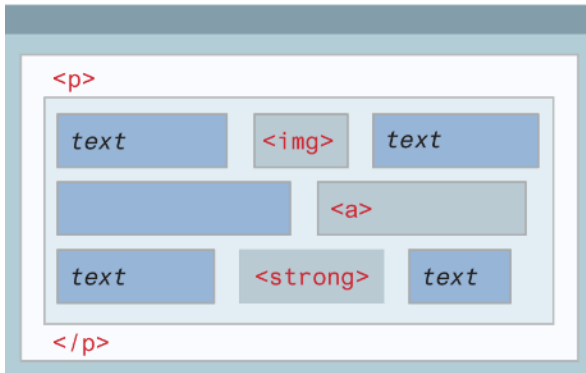
```
<p>  
This photo  of Conservatory Pond in  
<a href="http://www.centralpark.com">Central Park</a> was  
taken with a <strong>Canon EOS 30D</strong> camera.  
</p>
```



Inline content is laid out horizontally left to right within its container.

Once a line is filled with content, the next line will receive the remaining content, and so on.

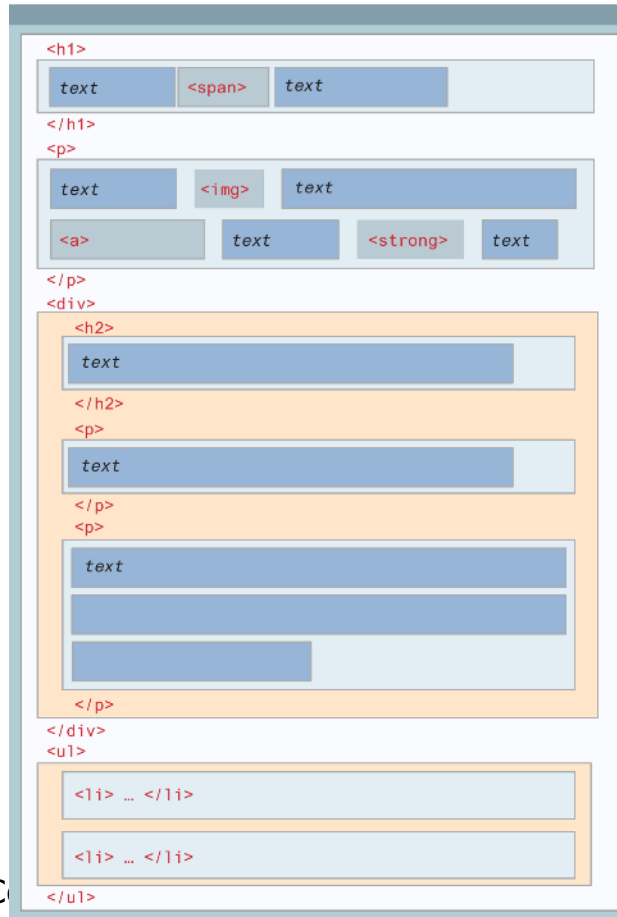
Here the content of this <p> element is displayed on two lines.



If the browser window resizes, then inline content will be "reflowed" based on the new width.

Here the content of this <p> element is now displayed on three lines.

Block and inline elements together



A document consists of block-level elements stacked from top to bottom.

Within a block, inline content is horizontally placed left to right.

Some block-level elements can contain other block-level elements (in this example, a `<div>` can contain other blocks).

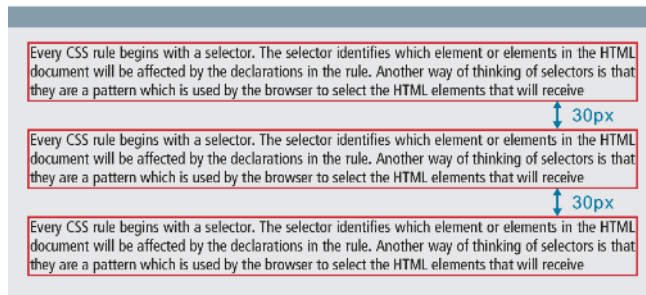
In such a case, the block-level content inside the parent is stacked from top to bottom within the container (`<div>`).

Margins and Padding

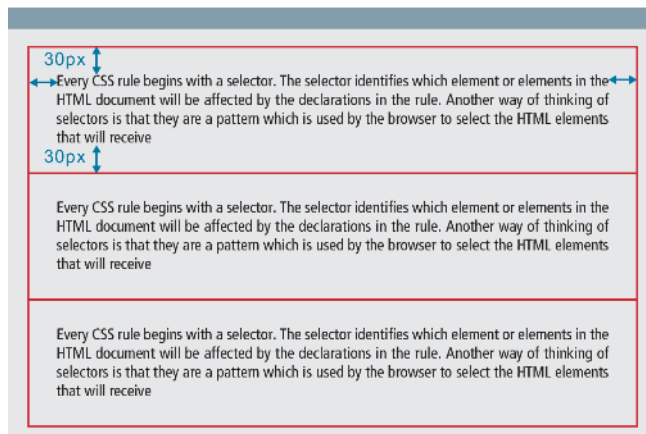
Margins add spacing around an element's content

Padding adds spacing within elements.

Borders divide the margin area from the padding area.



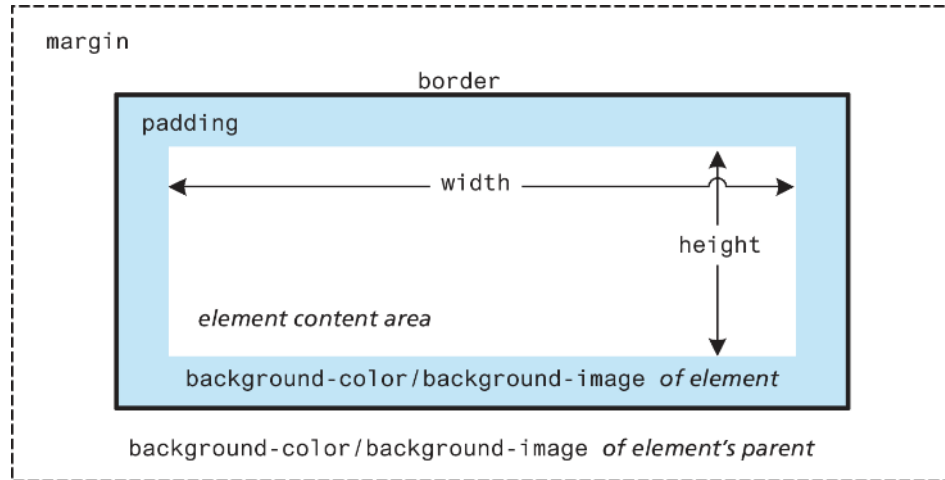
```
p {  
  border: solid 1pt red;  
  margin: 30px;  
  padding: 0;  
}
```



```
p {  
  border: solid 1pt red;  
  margin: 0;  
  padding: 30px;  
}
```

The Box Model

In CSS, all HTML elements exist within an **element box**. In order to become proficient with CSS, you must become familiar with the box model.



Box Model: All or one

NOTE

With border, margin, and padding properties, it is possible to set the properties for one or more sides in a single property, or individually

border-top-color: red; /* sets just the top side */

border-right-color: green; /* sets just the right side */

border-bottom-color: yellow; /* sets just the bottom side */

border-left-color: blue; /* sets just the left side */

Alternately, we can set all four sides to a single value via:

border-color: red; /* sets all four sides to red */

Or we can set all four sides to different values via:

border-color: red green orange blue;

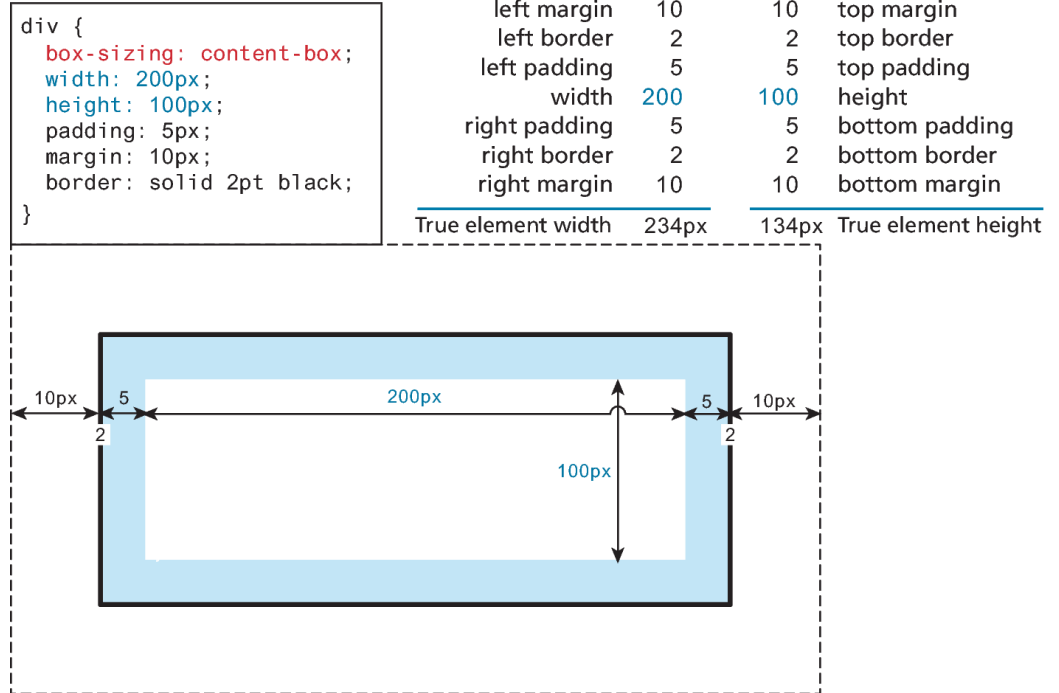
Another shortcut is to use just two values; in this case the first value sets top and bottom, while the second sets the right and left.

border-color: red yellow; /* top+bottom=red, right+left=yellow */



Box Dimensions

- Block-level elements have **width** and **height** properties.
- They also have a **min-width**, **min-height**, **max-width**, and **max-height** properties as well to help when the width or a height might be specified as a % of its parent container
- Since the width and the height only refer to the size of the content area, the **total size** of an element is equal to the size of its content area plus the sum of its padding, borders, and margins.



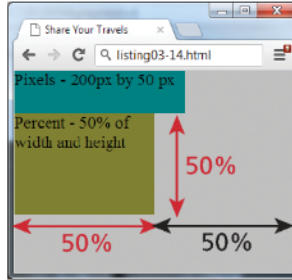
Overflow Property

[w3SchoolExample](https://www.w3schools.com/css/css_overflow.asp)

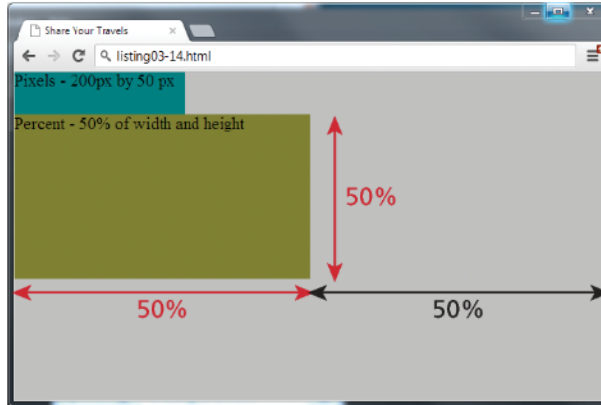
It is possible to control what happens with the content if the box's width and height are not large enough to display the content using the **overflow** property



```
<style>
  html, body {
    margin:0;
    width:100%;
    height:100%;
    background: silver;
  }
  .pixels {
    width:200px;
    height:50px;
    background: teal;
  }
  .percent {
    width:50%;
    height:50%;
    background: olive;
  }
</style>
```



```
<body>
  <div class="pixels">
    Pixels - 200px by 50 px
  </div>
  <div class="percent">
    Percent - 50% of width and height
  </div>
</body>
```



CSS Text Styling

CSS provides two types of properties that affect text.

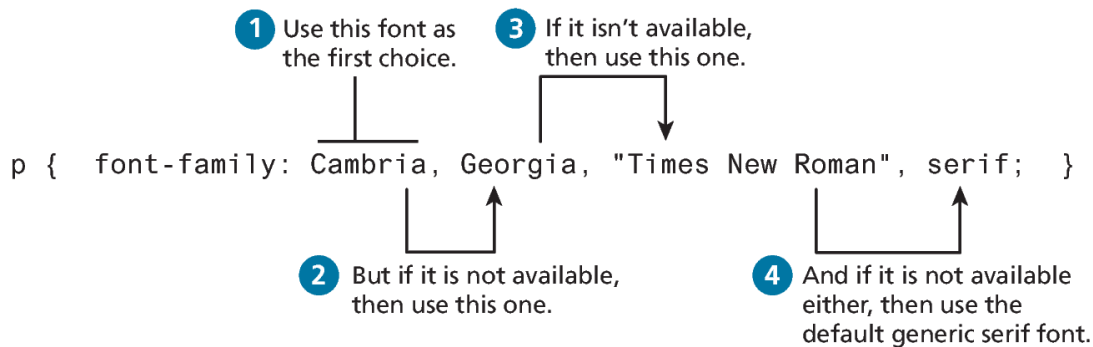
- **Font properties** that affect the font and its appearance.
- **Paragraph properties** that affect the text in a similar way no matter which font is being used
- Font properties include
 - font, font-family, font-style, font-size, font-variant, font-weight

Font Family

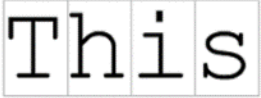
[Intervention] Slow network is detected. See [localhost/:1 https://www.chromestatus.com/feature/5636954674692096](https://www.chromestatus.com/feature/5636954674692096) for more details.
Fallback font will be used while loading:
<https://fonts.gstatic.com/s/lato/v24/S6uyw4BMUTPHjx4wXg.woff2>

Just because a given font is available on the web developer's computer does not mean it will be available for all users.

For this reason, it is conventional to supply a so-called **web font stack**— a series of alternate fonts (fallback fonts) to use in case the original font choice is not on the user's computer



Different font families

	Generic Font-Family Name	
This	serif	
This	sans-serif	 Without ("sans") serif
This	monospace	 In a monospace font, each letter has the same width.
This	cursive	 In a regular, proportionally-spaced font, each letter has a variable width.
This	fantasy	Decorative and cursive fonts vary from system to system; rarely used as a result.

@import a font available online

DIVE DEEPER

You can use a font even if it is not installed on the user's computer. Open source font sites such as Google Web Fonts (<https://fonts.google.com>) and Font Squirrel (<http://www.fontsquirrel.com/>) have online fonts available for public use:

To use Droid Sans, you can add the following to your <head> section

```
<link href="https://fonts.googleapis.com/css?family=Droid+Sans" rel="stylesheet" type="text/css">
```

Or, add the following import inside one of your CSS files:

```
@import url('https://fonts.googleapis.com/css2?family=Roboto+Slab:wght@400&display=swap');
```



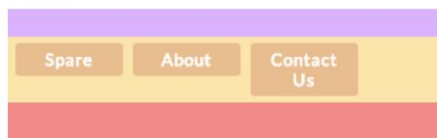


12

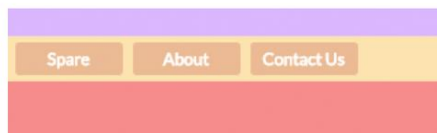


I've chosen to use one of the many Google Web Fonts as the main font for my website. The website is being displayed on both a Mac and Windows computer yet the font appears slightly different in terms of their dimensions and spacial conformations on the page.

Mac



Windows



intex (Member) asked a question.

August 13, 2016 at 9:50 AM

Which font is best for Mac/Windows compatibility?

Hi at all,

just wanted to ask which font to your experience is best for working on hybrid Mac/Windows/iOS/WebDirect solutions?

For years I worked with Verdana, but this font is slightly bigger on Windows than Mac, which sometimes breaks buttons and field names

Obviously it has to be a font that is existent on all OS. How about Arial, Helvetica and others?

Do you then take a different font for printing and screen layouts?

Thanks in advance

Martin

Design

Deployment

Upvote

Answer

Share

61 answers · 4.37K views



philmodjunk (Member)

7 years ago

Arial, Arial, Arial...

It's generally the most universally useful between Mac, iOS and Windows. Nicolas Hunter just recommended it for that reason at our most recent DIGFM meeting at FileMaker's headquarters in Santa Clara. I've used Times New Roman with few issues as well.

But all systems have different font metrics and the same text on Windows takes up more space than on a Mac. For that reason, I usually find that I have to do fewer layout tweaks if I

Font Sizes

If we wish to create web layouts that work well on different devices, we should learn to use relative units such as **em units** or **percentages** for our font sizes

- it can quickly become difficult to calculate actual sizes when there are nested elements
 - For this reason, CSS3 now supports a new relative measure, the **rem** (for root em unit). This unit is always relative to the size of the root element (i.e., the `<html>` element).


```
font-weight: normal|bold|bolder|lighter|number|initial|inherit;
```

Property Values

Value	Description
normal	Defines normal characters. This is default
bold	Defines thick characters
bolder	Defines thicker characters
lighter	Defines lighter characters
100 200 300 400 500 600 700 800 900	Defines from thin to thick characters. 400 is the same as normal, and 700 is the same as bold
initial	Sets this property to its default value. Read about <i>initial</i>

Text properties

Every CSS rule begins with a selector. The selector identifies which element or elements in the HTML document will be affected by the declarations in the rule.

`line-height: normal;`

Every CSS rule begins with a selector. The selector identifies which element or elements in the HTML document will be affected by the declarations in the rule.

`line-height: 1.5;`

Every CSS rule begins with a selector. The selector identifies which element or elements in the HTML document will be affected by the declarations in the rule.

`text-indent: 4em;`

Every CSS rule begins with a selector.

`text-align: left;`

Every CSS rule begins with a selector.

`text-align: center;`

Every CSS rule begins with a selector.

`text-align: right;`

Every CSS rule begins with a selector.

`letter-spacing: 3px;`



`vertical-align: top;`



`vertical-align: middle;`



`vertical-align: bottom;`

EVERY RULE BEGINS WITH A SELECTOR

`text-transform: uppercase;`

every rule begins with a selector

`text-transform: lowercase;`

selector

`text-decoration: underline;`

selector

`text-decoration: none;`

Example

CSS Variables

CSS Variables (also called custom properties) are a new feature that let you

- define variables (which must begin with a double hyphen --x) at the top of your CSS file usually within a special **:root** pseudo-class selector, and
- reference those variables as property values using the **var()** CSS function.

Duplication of colors, spacing, and borders, often happens numerous times within a single file.

Example Part 1 (duplicate values)

```
header {  
  background-color: #431c5d;  
  color: #e05915;  
  padding: 4px;  
  box-shadow: 6px 5px 20px 1px rgba(0,0,0,0.22);  
  margin: 0;  
}  
header button {  
  background-color: #e05915;  
  border-radius: 5px;  
  border-color: #e6e9f0;  
  padding: 4px;  
  color: #e6e9f0;  
  font-size: 18px;  
  margin-top: 9px;  
}  
  
#results {  
  background-color: #431c5d;  
  font-size: 18px;  
  border-radius: 5px;  
  padding: 4px;  
  box-shadow: 6px 5px 20px 1px rgba(0,0,0,0.22);  
}
```

LISTING 4.8 Duplicate property values in CSS

Example Part 2 (CSS variables)

```
:root {
  --bg-color-main: #431c5d;
  --bg-color-secondary: #e05915;
  --fg-color-main: #e6e9f0;
  --radius-boxes: 5px;
  --padding-boxes: 4px;
  --font-size-default: 18px;
  --shadow-color: rgba(0,0,0,0.22);
  --dropshadow: 6px 5px 20px 1px var(--shadow-color);
}

header {
  background-color: var(--bg-color-main);
  color: var(--bg-color-secondary);
  padding: var(--padding-boxes);
  box-shadow: var(--dropshadow);
  margin: 0;
}

header button {
  background-color: var(--bg-color-secondary);
  border-radius: var(--radius-boxes);
  border-color: var(--fg-color-main);
  padding: var(--padding-boxes);
  color: var(--fg-color-main);
  font-size: var(--font-size-default);
  margin-top: calc( --font-size-default / 2 );
}

#results {
  background-color: var(--bg-color-main);
  font-size: var(--font-size-default);
  border-radius: var(--radius-boxes);
  padding: var(--padding-boxes);
  box-shadow: var(--dropshadow);
}
```

LISTING 4.9 Using CSS Variables

Styling Tables (Borders)

Borders can be assigned to the **<table>**, **<th>** and the **<td>** element. Interestingly, borders cannot be assigned to the **<tr>**, **<thead>**, **<tfoot>**, and **<tbody>** elements.

Notice as well the **border-collapse** property. This property selects the table's border model.

- The default is the **separated** model or value (each cell has its own unique borders)
- The **collapsed** border model makes adjacent cells share a single border.

Styling table borders

Chapter 5

figure05-08.html

19th Century French Paintings

Title	Artist	Year
The Death of Marat	Jacques-Louis David	1793
Burial at Omans	Gustave Courbet	1849
The Sleepers	Gustave Courbet	1860
Liberty Leading the People	Eugene Delacroix	1830
Total Number of Paintings		4

```
table {
  border: solid 1pt black;
}
```

figure05-08.html

19th Century French Paintings

Title	Artist	Year
The Death of Marat	Jacques-Louis David	1793
Burial at Omans	Gustave Courbet	1849
The Sleepers	Gustave Courbet	1860
Liberty Leading the People	Eugene Delacroix	1830
Total Number of Paintings		4

```
table {
  border: solid 1pt black;
}
td {
  border: solid 1pt black;
}
```

Chapter 5

figure05-08.html

19th Century French Paintings

Title	Artist	Year
The Death of Marat	Jacques-Louis David	1793
Burial at Omans	Gustave Courbet	1849
The Sleepers	Gustave Courbet	1860
Liberty Leading the People	Eugene Delacroix	1830
Total Number of Paintings		4

```
table {
  border: solid 1pt black;
  border-collapse: collapse;
}
td {
  border: solid 1pt black;
}
```

Chapter 5

figure05-08.html

19th Century French Paintings

Title	Artist	Year
The Death of Marat	Jacques-Louis David	1793
Burial at Omans	Gustave Courbet	1849
The Sleepers	Gustave Courbet	1860
Liberty Leading the People	Eugene Delacroix	1830
Total Number of Paintings		4

```
table {
  border: solid 1pt black;
  border-collapse: collapse;
}
td {
  border: solid 1pt black;
  padding: 10pt;
}
```

19th Century French Paintings

Title	Artist	Year
The Death of Marat	Jacques-Louis David	1793
Burial at Omans	Gustave Courbet	1849
The Sleepers	Gustave Courbet	1860
Liberty Leading the People	Eugene Delacroix	1830
Total Number of Paintings		4

```
table {
  border: solid 1pt black;
  border-spacing: 10pt;
}
td {
  border: solid 1pt black;
}
```

Boxes and Zebras

While there is almost no end to the different ways one can style a table, there are a number of common approaches. We will look at two of them here.

- The first of these is a box format, in which we simply apply background colors and borders in various ways



Title	Artist	Year
The Death of Marat	Jacques-Louis David	1793
Burial at Ornans	Gustave Courbet	1849
The Sleepers	Gustave Courbet	1860
Liberty Leading the People	Eugene Delacroix	1830
Mademoiselle Caroline Riviere	Jean-Auguste-Dominique Ingres	1806

```
caption {
  font-weight: bold;
  padding: 0.25em 0 0.25em 0;
  text-align: left;
  text-transform: uppercase;
  border-top: 1px solid #DCA806;
}
table {
  font-size: 0.8em;
  font-family: Arial, sans-serif;
  border-collapse: collapse;
  border-top: 4px solid #DCA806;
  border-bottom: 1px solid white;
  text-align: left;
}
```



Title	Artist	Year
The Death of Marat	Jacques-Louis David	1793
Burial at Ornans	Gustave Courbet	1849
The Sleepers	Gustave Courbet	1860
Liberty Leading the People	Eugene Delacroix	1830
Mademoiselle Caroline Riviere	Jean-Auguste-Dominique Ingres	1806

```
thead tr {
  background-color: #CACACA;
}
th {
  padding: 0.75em;
}
```



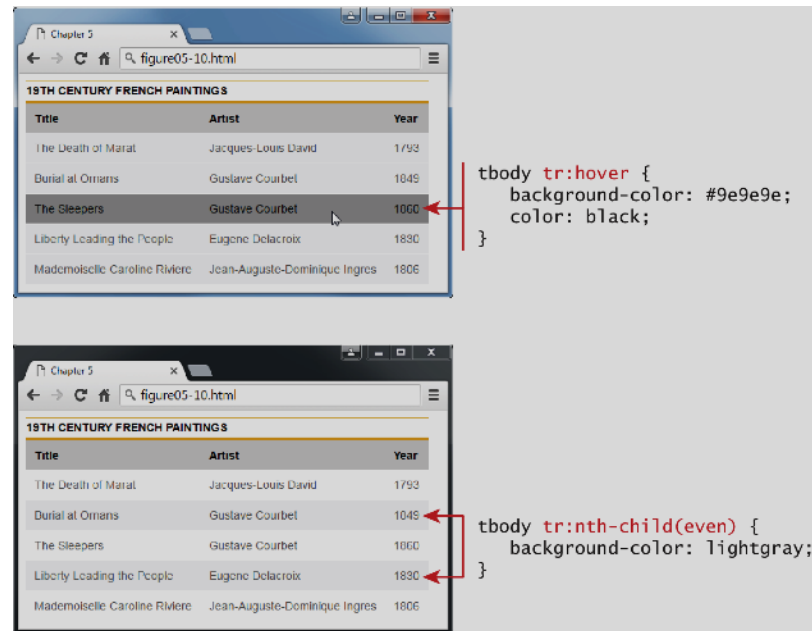
Title	Artist	Year
The Death of Marat	Jacques-Louis David	1793
Burial at Ornans	Gustave Courbet	1849
The Sleepers	Gustave Courbet	1860
Liberty Leading the People	Eugene Delacroix	1830
Mademoiselle Caroline Riviere	Jean-Auguste-Dominique Ingres	1806

```
tbody tr {
  background-color: #F1F1F1;
  border-bottom: 1px solid white;
  color: #6E6E6E;
}
tbody td {
  padding: 0.75em;
}
```


Boxes and Zebras (ii)

We can then

- add special styling to the **:hover** pseudo-class of the `<tr>` element to highlight a row when the mouse cursor hovers over
- use the **pseudo-element nth-child** to alternate the format of every second row.



19TH CENTURY FRENCH PAINTINGS

Title	Artist	Year
The Death of Marat	Jacques-Louis David	1793
Burial at Ornans	Gustave Courbet	1845
The Sleepers	Gustave Courbet	1866
Liberty Leading the People	Eugene Delacroix	1830
Mademoiselle Caroline Riviere	Jean-Auguste-Dominique Ingres	1806

```
tbody tr:hover {  
  background-color: #9e9e9e;  
  color: black;  
}
```

19TH CENTURY FRENCH PAINTINGS

Title	Artist	Year
The Death of Marat	Jacques-Louis David	1793
Burial at Ornans	Gustave Courbet	1845
The Sleepers	Gustave Courbet	1866
Liberty Leading the People	Eugene Delacroix	1830
Mademoiselle Caroline Riviere	Jean-Auguste-Dominique Ingres	1806

```
tbody tr:nth-child(even) {  
  background-color: lightgray;  
}
```

Styling Form Elements

Let's begin with the common text and button controls. A common styling change is to eliminate the borders and add in rounded corners and padding.

Default control appearance

placeholder Click

Customized controls

placeholder placeholder

Click Secondary A Secondary B

```
border: 0;
margin: 3px;
padding: 7px 5px;
border-radius: 2px;
```

```
::placeholder {
  color: #D9E2EC;
}
```

```
border-bottom: 1px solid;
```

```
border: 0;
margin: 4px;
padding: 5px;
color: white;
width: 100px;
cursor: pointer;
border-radius: 2px;
background-color: #003E6B;
```

```
...
border-radius: 5px;
background-color: #F0F4F8;
color: #003E6B;
border: solid 1pt #9FB3C8;
box-shadow: 4px 4px 8px
            rgba(159,179,200,.7);
```

pseudo element
that allows you to style the
placeholder text of a form
element.

[Example](https://www.w3schools.com/cssref/css_selectors.asp)

[https://www.w3schools.com/cssref/
css_selectors.asp](https://www.w3schools.com/cssref/css_selectors.asp)

Working with labels

Multiple controls on single line

Title Year

```
<label for="title">Title</label>
<input type="text" name="title" id="title"/>
<label for="year">Year</label>
<input type="text" name="year" id="year" />
```

Each label and control pair on single line

Title
Year

Requires CSS layout techniques, such as grid, flex, floats, or positioning.

```
label {
  display: block;
  margin: 10px 0 0 5px;
}
```

Vertical

Title
Year

Vertical, but no labels

Title
Year

```
<input type="text" name="title" placeholder="Title" />
<input type="text" name="year" placeholder="Year" />
```

Form Design

A well-designed form communicates to a user that the site values their time and data.

Phone #:

Address:

City:

Region:
Riyadh

Zip Code:

Website or Domain Name:

Do you have hosting?
☒ Yes
☐ No

Project Description:

Submit Reset

Account Settings

User

Email
a@bb

Change Password

Language
English

Profile

First Name
Randy

Last Name
Connolly

About
Is stressed to be writing about design

Notifications

By Email

☒ For comments

☒ For special offers

By Text Messages

☒ Same as Email

☐ No text messages

Save Settings Cancel

Use a background color to add contrast between input controls and rest of page.

Use placeholders to indicate input format.

Buttons should be used for actions.

Group related items in visually distinct sections.

Use borders to separate sections.

Add space above labels to make it clear to which control the label is connected.

Add white space within form controls by increasing height and adding padding.

Indicate which control has focus.

Style select lists, checkboxes, and radio buttons differently from the default look.

Use size, typography, and color to emphasize/deemphasize relative importance of information or controls.

The primary action button should be clearly indicated with the largest visual contrast on form.

Secondary buttons should be less prominent with lower contrast or as outlines.

[Source code: chapter03/07_style_forms](#)

Third Edition by Randy Connolly and Ricardo Hoar

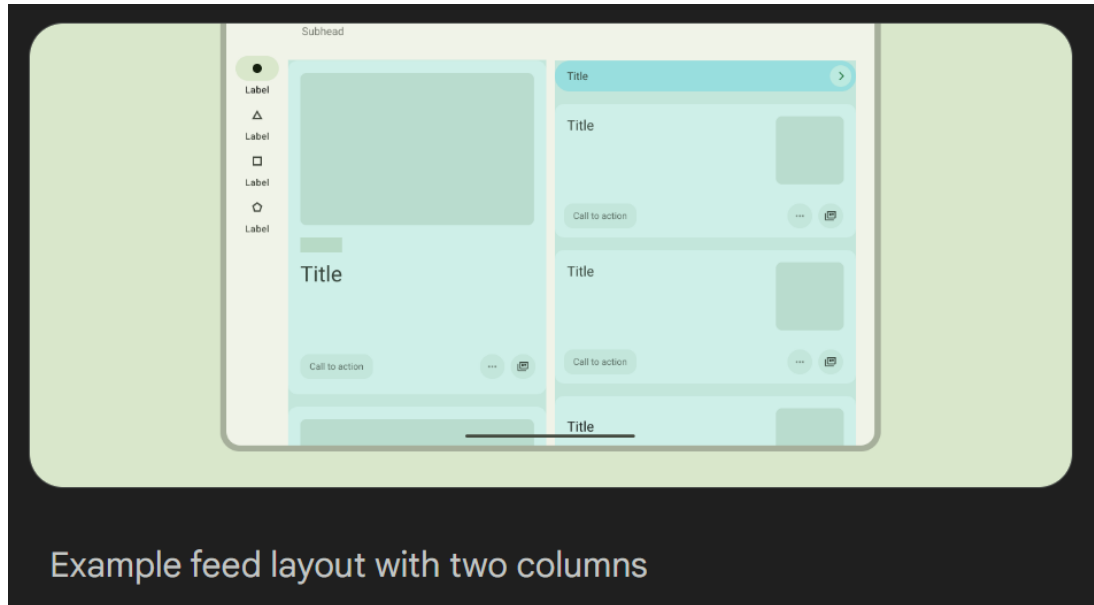


Chapter 2

CSS Part 2: Layout

In this chapter you will learn . . .

- Approaches to CSS layout using CSS flexbox and grid models
- What responsive web design is and how to construct responsive designs



To learn more about page layout design, this resource is comprehensive:

<https://m3.material.io/>

Flexbox Layout (for 1D layouts)

Note: In the past tables were sometimes used to design a page's layout. This practice should never be used:

- It pollutes the HTML,
- makes things more difficult for SEO
- and is not maintainable.

← float



Fall in Calgary

Nunc nec fermentum dolor. Duis at iaculis turpis. Sed rutrum elit ac egestas dapibus. Duis nec consequat enim.

Mauris porta arcu id magna adipiscing lacinia at congue lacus. Vivamus blandit quam quis tincidunt egestas. Etiam posuere lectus sed sapien malesuada molestie.

Phasellus vel felis purus. Aliquam consequat pellentesque dui, non mollis erat dictum sit amet. Curabitur non quam dictum, consectetur arcu in, vehicula justo.

margin-left

= image size + margin-right

```
.media-image {  
  float: left;  
  margin-right: 10px;  
}  
.media-body {  
  margin-left: 160px;  
}
```

```
<div class="media">  
    
  <div class="media-body">  
    <h2>Fall in Calgary</h2>  
    <p>Nunc nec fermentum dolor...</p>  
    <p>Mauris porta arcu id...</p>  
    <p>Phasellus vel felis purus...</p>  
  </div>  
</div>
```

Prior to flexbox, one would create such a layout within a container using floats plus margins. The problem with this approach is that margins needed to be in pixels and had to exactly match image size. If image size changed (or you wanted same kind of style elsewhere), you had to modify the style.



Fall in Calgary

Nunc nec fermentum dolor. Duis at iaculis turpis. Sed rutrum elit ac egestas dapibus. Duis nec consequat enim.

Mauris porta arcu id magna adipiscing lacinia at congue lacus. Vivamus blandit quam quis tincidunt egestas. Etiam posuere lectus sed sapien malesuada molestie.

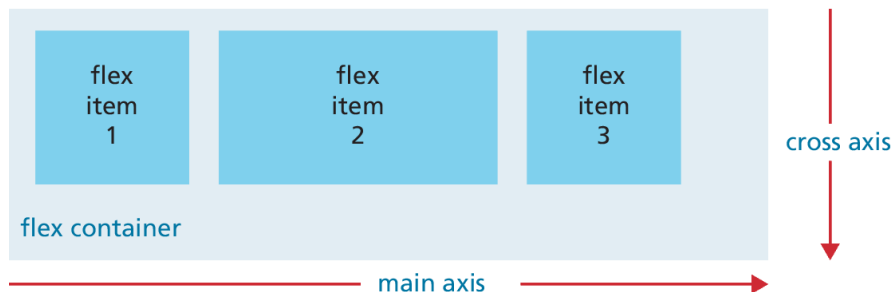
Phasellus vel felis purus. Aliquam consequat pellentesque dui, non mollis erat dictum sit amet. Curabitur non quam dictum, consectetur arcu in, vehicula justo.

```
.media {  
  display: flex;  
  align-items: flex-start;  
}  
.media-image {  
  margin-right: 1em;  
}
```

Using flexbox, we now have a much more generalized (and thus reusable) style.

Flex containers and items

There are two places in which you will be assigning flexbox properties: the **flex container** and the **flex items** within the container.



```
<div class="container">  
  <div>flex item 1</div>  
  <span>flex item 2</span>  
  <div>flex item 3</div>  
</div>
```

Both block-level and inline elements will become flex items.

[W3school Example](#)

The flexbox container properties

display: **flex** Necessary to use flexbox

flex-direction: **row**



Specifies the direction of the main axis.
The default is **row**.

flex-direction: **row-reverse**

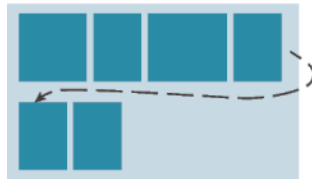


flex-direction: **column**

flex-direction: **column-reverse**



flex-wrap: **wrap**



flex-wrap: **nowrap**



Indicates whether new lines should be created if flex container can't contain all the flex items.

[Exmple: flex_intro.html](https://example.com/flex_intro.html)

The flexbox container properties (ii)

`justify-content: flex-start`

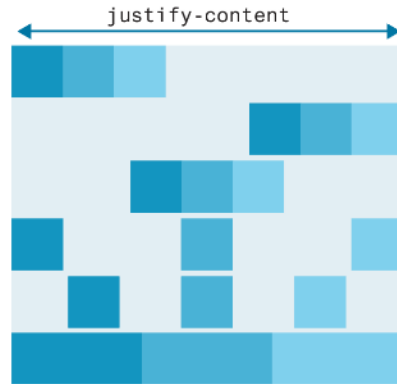
`justify-content: flex-end`

`justify-content: center`

`justify-content: space-between`

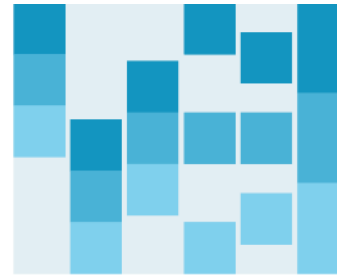
`justify-content: space-around`

`justify-content: stretch`



Controls flex item alignment along main axis.

align-items



Controls flex item alignment along cross axis (i.e., wrapped items).

`align-items: stretch`
`align-items: space-around`
`align-items: center`
`align-items: flex-end`
`align-items: flex-start`

The flexbox item (child) properties

You can also assign a fraction to each child item.

flex-basis: **auto**



The flex-basis property determines the initial size of the flex item before the remaining space is distributed.

The default auto value means that the size is determined by the width and height.

flex-basis: **200px**



You can specify a width using px, %, or other measurement units.

flex-grow: **2**



Defines the growth factor of an element relative to the other items.

$\text{width} = n$

$\text{width} = n \times 2$

flex-shrink: **2**



Defines how much an item will shrink when not enough space in container.

[Exmple: flex_basis.html](#)

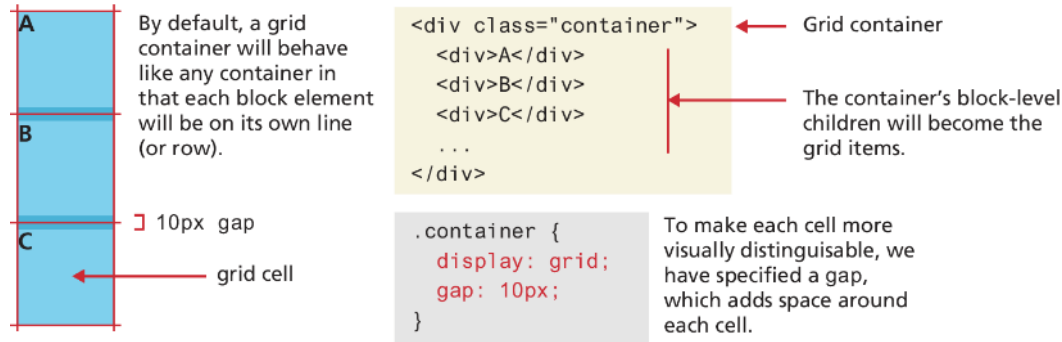
[Example](#)

```
.flex-item-1 {  
  flex: 1;  
}  
  
.flex-item-2 {  
  flex: 2;  
}  
  
.flex-item-3 {  
  flex: 3;  
}  
  
.flex-item-4 {  
  flex: 4;  
}  
  
.flex-item-6 {  
  flex: 6;  
}
```

Grid Layout (for 2D layouts)

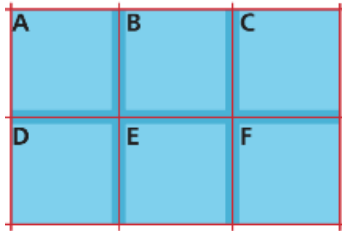
Grid layout is adjustable, powerful, and, compared to floats and even flexbox, is relatively easy to learn and use!

- Each block-level child in a parent container whose **display** property is set to **grid** will be automatically placed into a grid cell



Specifying Grid Structure

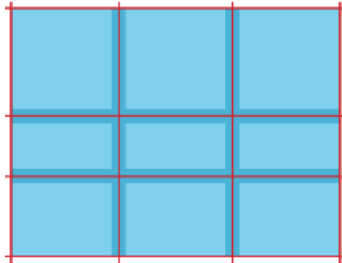
grid-template-columns is used for adding columns by specifying each column's width using the **fr** unit.



```
.container {  
  display: grid;  
  gap: 10px;  
  grid-template-columns: 1fr 1fr 1fr;  
}
```

This makes each column equal to an equal fraction of the available space.

You can specify the number of columns per row/line via the grid-template-column property.



```
.container {  
  ...  
  height: 300px;  
  grid-template-columns: 1fr 1fr 1fr;  
  grid-template-rows: 80px 30px 50px;  
}
```

While you often do not need to set a row height, you can make rows taller than their actual content.

Specifying column widths

Column widths can be specified

The CSS **repeat()** function provides a way to specify repeating patterns of columns.

```
grid-template-columns: 70px 70px 45px;
```



Each column can have its own unique width value, in px, %, em, etc.

```
grid-template-columns: 50px auto 50px;
```



An auto value indicates the width will fill the remaining space.

```
grid-template-columns: repeat(3, 1fr);
```



The repeat function can be used to defining a repeating pattern.

```
grid-template-columns: repeat(2, 30px 50px) 70px;
```



30px 50px 30px 50px 70px

Specifying column widths (ii)



```
grid-template-columns: repeat(auto-fill, minmax(100px, 1fr));
```

Fills the row with as many columns as can fit into the container space.

The min column size is 100px and the max is whatever size is necessary to allow each column to be equal sized.



```
grid-template-columns: repeat(auto-fit, minmax(100px, 1fr));
```

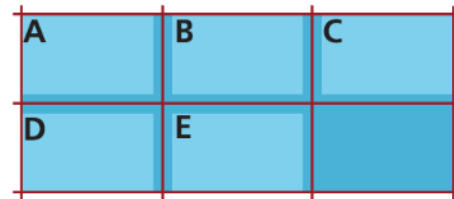
Fits the columns into space by expanding the column width into the available container space.

[Example](#)

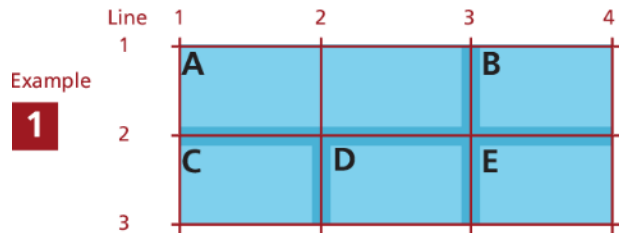
Explicit Grid Placement Example 1

```
<div class="container">
  <div class="a">A</div>
  <div class="b">B</div>
  <div class="c">C</div>
  <div class="d">D</div>
  <div class="e">E</div>
</div>
```

```
.container {
  display: grid;
  gap: 10px;
  grid-template-columns: repeat(3,1fr);
  grid-template-rows: repeat(2,200px);
}
```



With implicit layout, grid items are placed automatically.



```
.a {
  grid-column-start: 1;
  grid-column-end: 3;
}
```

The start and end numbers refer to the line number not the column number.

The same effect also possible using either of the following:

```
grid-column: 1 / 3;
```

```
grid-column: 1 / span 2;
```

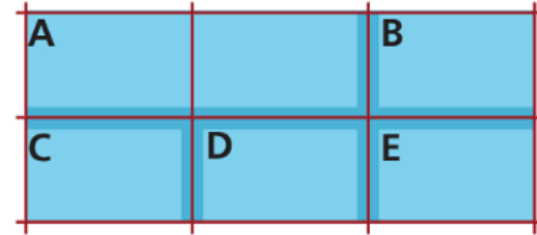
[w3SchoolsExample](#) (grid column)

[w3SchoolsExample](#) (grid row)

Explicit Grid Placement Example 2

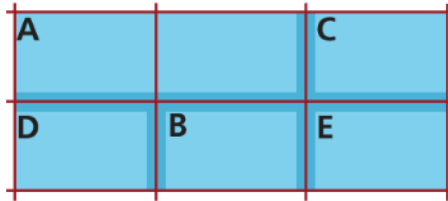
```
<div class="container">  
  <div class="a">A</div>  
  <div class="b">B</div>  
  <div class="c">C</div>  
  <div class="d">D</div>  
  <div class="e">E</div>  
</div>
```

```
.container {  
  display: grid;  
  gap: 10px;  
  grid-template-columns: repeat(3,1fr);  
  grid-template-rows: repeat(2,200px);  
}
```



With implicit layout, grid items are placed automatically.

2

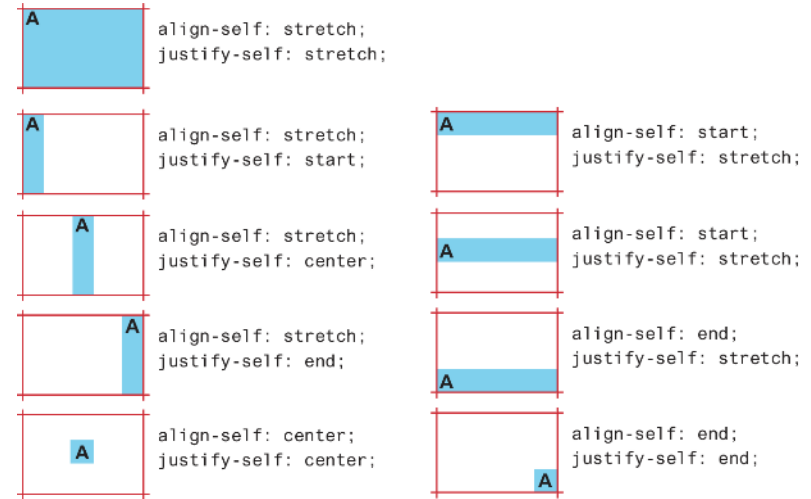


```
.b {  
  grid-row: 2;  
  grid-column: 2;  
}
```

Grid cells can be placed into any row and column.

Cell properties

- **align-self** and **justify-self** control the cell content's horizontal and vertical alignment within its grid container.

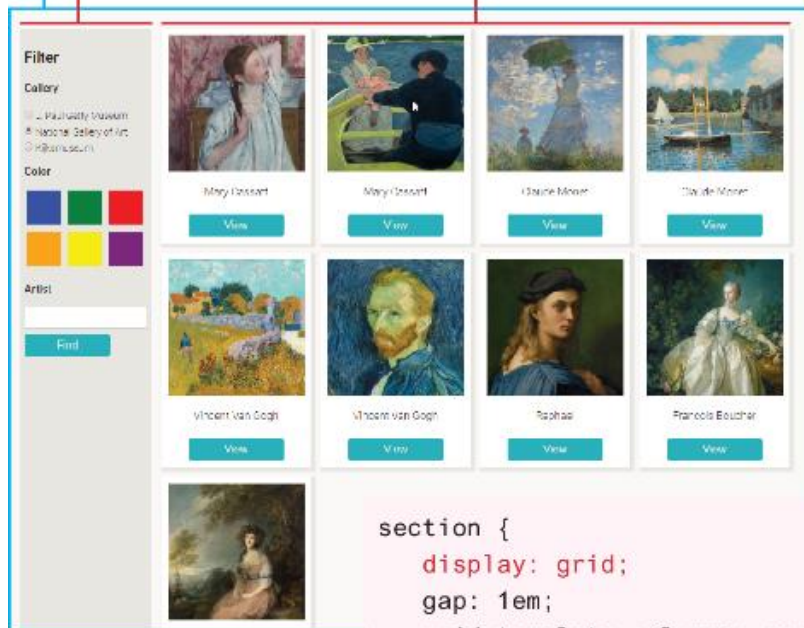


- You can similarly control cell alignment within a grid container using **align-items** and **justify-items**

Nested Grid

```
main {  
  display: grid;  
  grid-template-columns: 1fr 4fr;  
}
```

```
<main>  
  <aside>  
    <h2>Filter</h2>  
    ...  
  </aside>  
  <section>  
    <div class="card">...  
    <div class="card">...  
    ...  
  </section>  
</main>
```



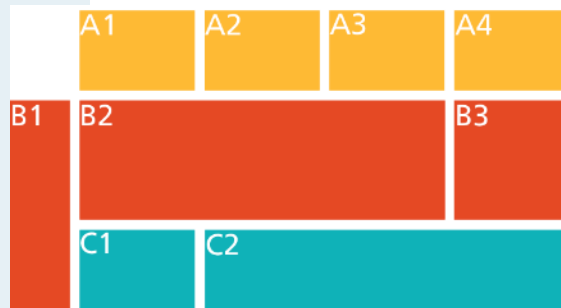
```
section {  
  display: grid;  
  gap: 1em;  
  grid-template-columns: repeat(auto-fit, 220px);  
}
```

Example

Grid Named Areas

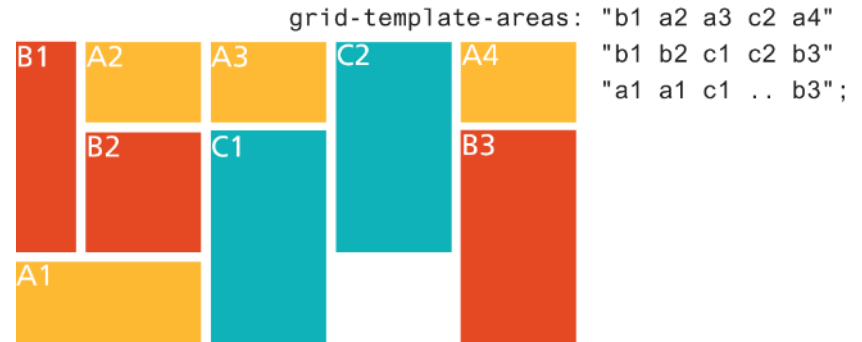
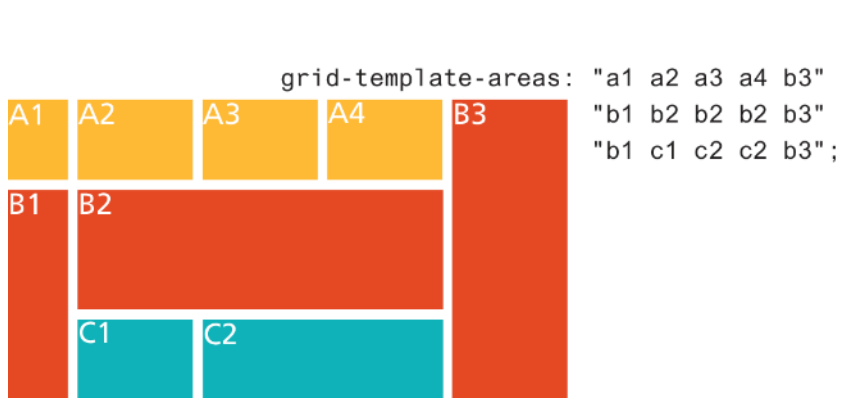
```
<style>
.container {
  grid-gap: 10px;
  display: grid;
  grid-template-rows: 100px 150px 100px;
  grid-template-columns: 75px 1fr 1fr 1fr 1fr;
  grid-template-areas: ". a1 a2 a3 a4"
                      "b1 b2 b2 b2 b3"
                      "b1 c1 c2 c2 c2";
}
.a1 { grid-area: a1; }
.a2 { grid-area: a2; }
.a3 { grid-area: a3; }
.a4 { grid-area: a4; }
.b1 { grid-area: b1; }
.b2 { grid-area: b2; }
.b3 { grid-area: b3; }
.c1 { grid-area: c1; }
.c2 { grid-area: c2; }
</style>
```

```
<section class="container">
  <div class="yellow a1">A1</div>
  <div class="yellow a2">A2</div>
  <div class="yellow a3">A3</div>
  <div class="yellow a4">A4</div>
  <div class="orange b1">B1</div>
  <div class="orange b2">B2</div>
  <div class="orange b3">B3</div>
  <div class="cyan c1">C1</div>
  <div class="cyan c2">C2</div>
</section>
```



LISTING 7.2 Using grid areas

Grid Areas (ii)



LISTING 7.2 Using grid areas

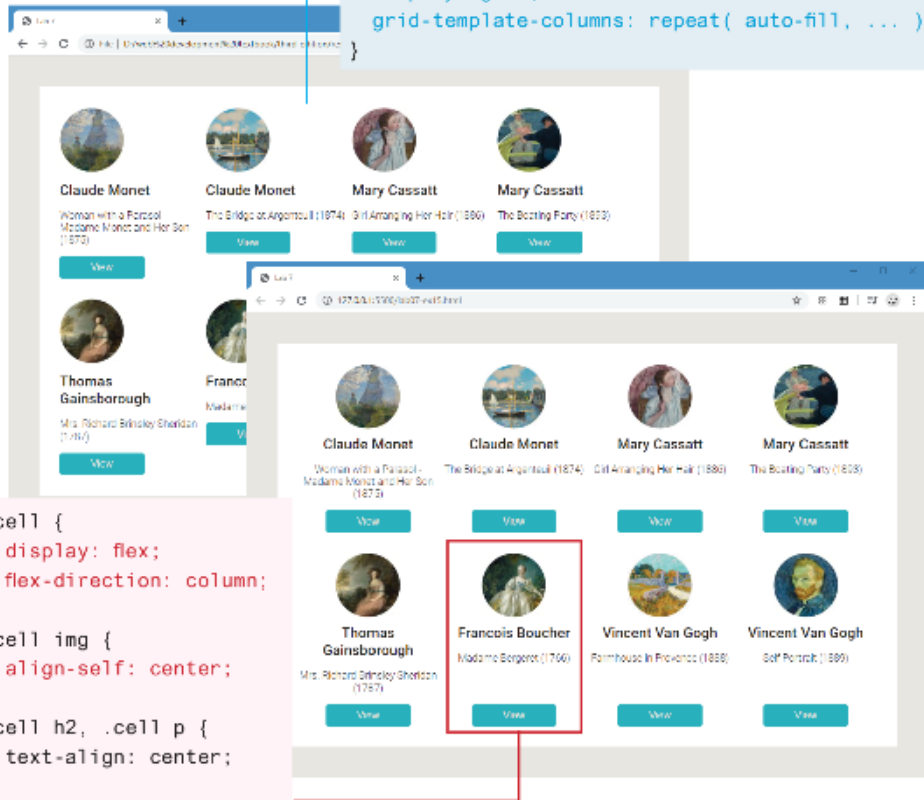
[w3schoolsExample](#)

Grid and Flexbox Together

- **grid** and **flexbox** each have their strengths and these strengths can be combined
- **grid** layout is ideal for constructing the layout structure of your page
- **flexbox** is ideal for laying out the contents of a grid cell.

Grid layout is used to create row and column grid.

```
.container {  
  display: grid;  
  grid-template-columns: repeat( auto-fill, ... );  
}
```



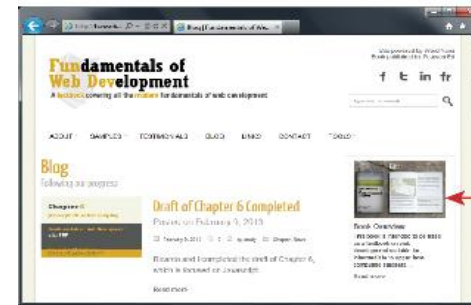
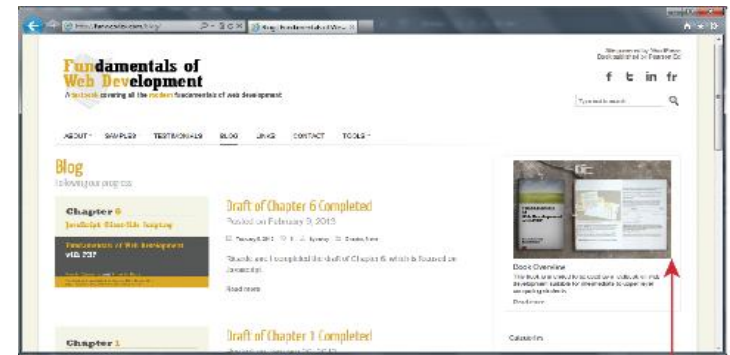
```
.cell {  
  display: flex;  
  flex-direction: column;  
}  
.cell img {  
  align-self: center;  
}  
.cell h2, .cell p {  
  text-align: center;  
}  
.cell button {  
  margin-top: auto;  
  justify-self: flex-end;  
  align-self: center;  
}
```

To layout each individual grid cell, we will need to use flexbox.

Responsive Design

In a **responsive design**, the page “responds” to changes in the browser size that go beyond simple percentage scaling of widths.

- smaller images will be served and
- navigation elements will be replaced as the browser window shrinks



Notice how some elements are scaled to shrink as browser window reduces in size.



When browser shrinks below a certain threshold, then layout and navigation elements change as well.

In this case, the `<ul>` list of hyperlinks changes to a `<select>` and the two-column design changes to one column.

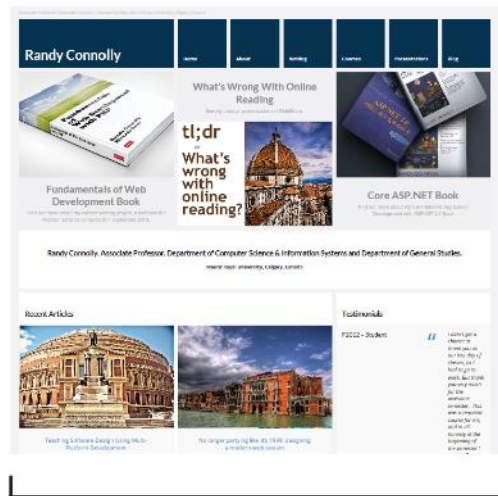
Viewports

The browser's **viewport** is the part of the browser window that displays web content.

Mobile browsers will by default scale a web page down to fit the width of the screen.

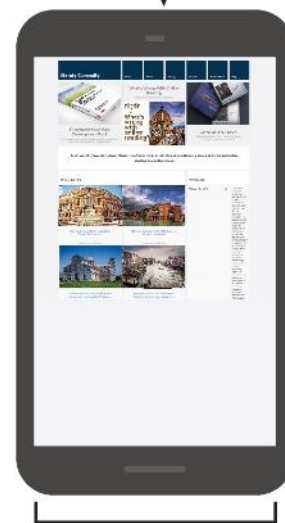
Generally, results in a viewing experience that works but is very difficult to read and use.

1 Mobile browser renders web page on its viewport



960px
Mobile browser viewport

2 It then scales the viewport to fit within its actual physical screen



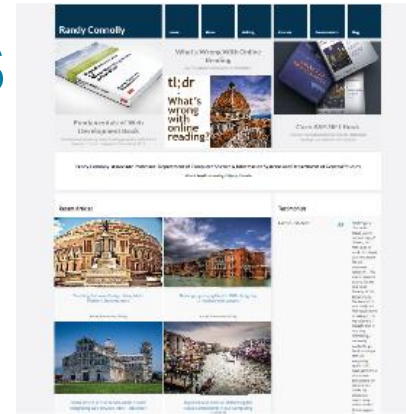
320px
Mobile browser screen

Setting Viewports

```
<html>
<head>
<meta name="viewport"
content="width=device-width, initial-scale=1" />
```

Fig, Setting the Viewport

The **width** attribute controls the size of the viewport, while **initial-scale** sets the zoom level.



```
<meta name="viewport"
content="width=device-width, initial-scale=1" />
```

- 1 Mobile browser renders web page on its viewport and because of the <meta> setting, makes the viewport the same size as the pixel size of screen.



320px
Mobile browser viewport



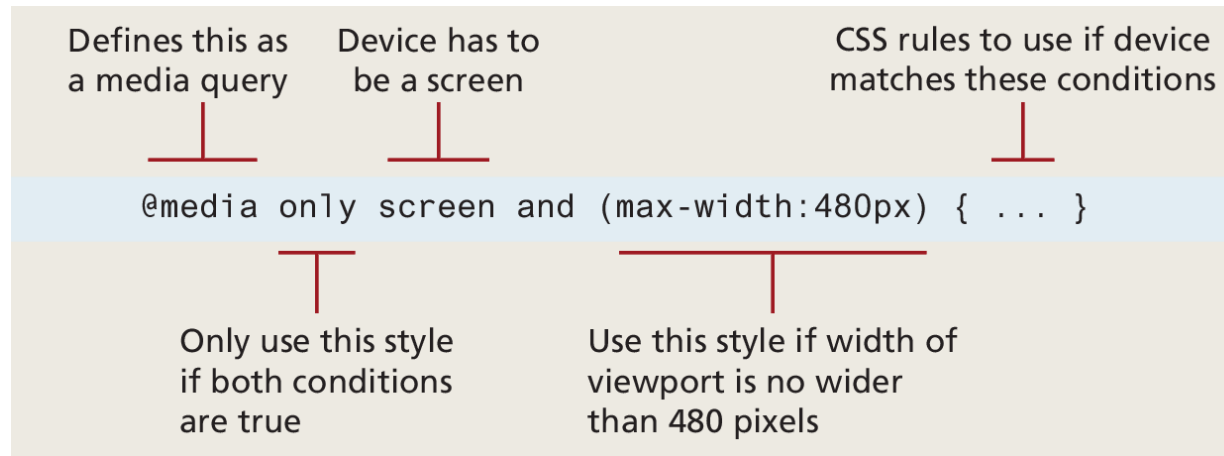
320px

- 2 It then displays it on its physical screen with no scaling.

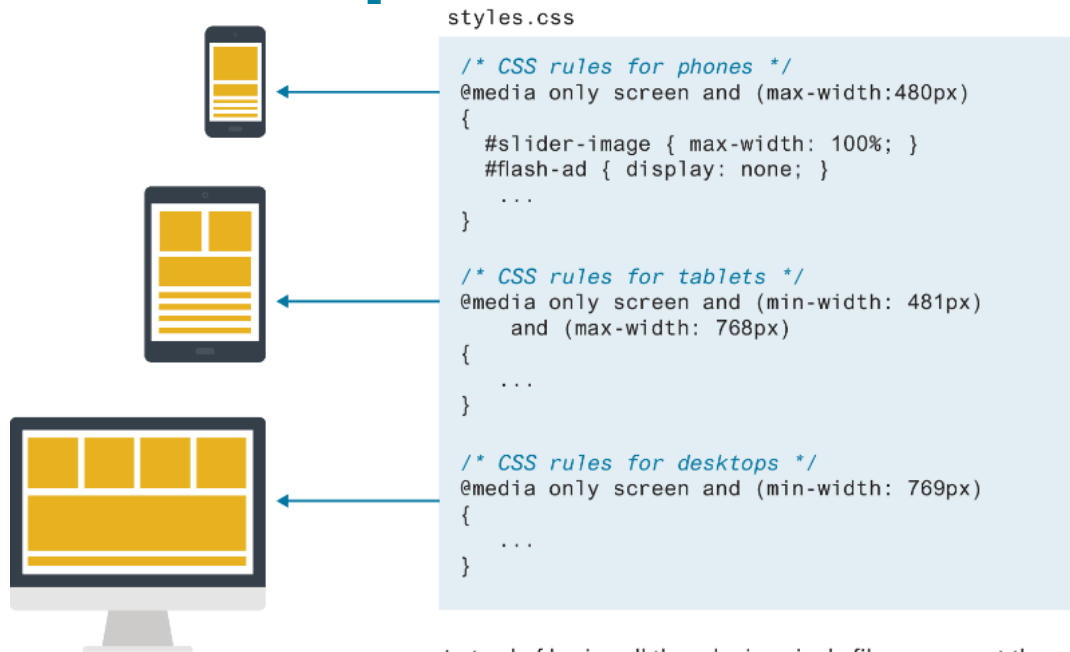
Media Queries

CSS media queries are a way to apply style rules based on the medium that is displaying the file

Contemporary responsive sites will typically provide CSS rules for phone displays first, then tablets, then desktop monitors, an approach called **progressive enhancement**



Media queries in action



Instead of having all the rules in a single file, we can put them in separate files and add media queries to `<link>` elements.

```
<link rel="stylesheet" href="mobile.css" media="screen and (max-width:480px)" />
<link rel="stylesheet" href="tablet.css"
      media="screen and (min-width:481px) and (max-width:768px)" />
<link rel="stylesheet" href="desktop.css" media="screen and (min-width:769px)" />
```

Window class (width)	Breakpoint (dp)	Common devices
Compact	Width < 600	Phone in portrait
Medium	600 ≤ width < 840	Tablet in portrait Foldable in portrait (unfolded)
Expanded	840 ≤ width < 1200*	Phone in landscape Tablet in landscape Foldable in landscape (unfolded) Desktop
Large	1200 ≤ width < 1600	Desktop
Extra-large	1600 ≤ width	Desktop Ultra-wide

Fig. Breakpoints in [material design](#) 3.0

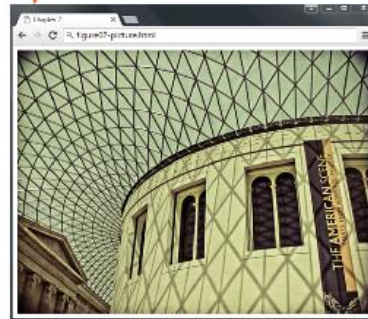
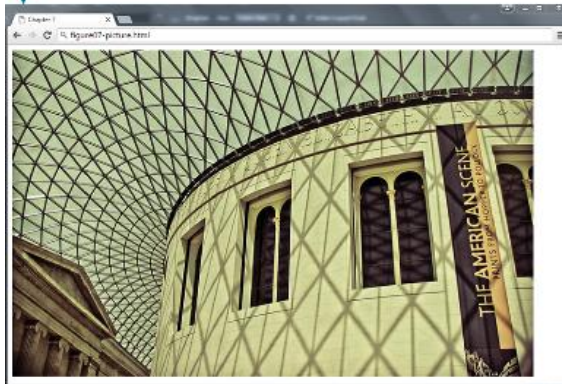
The <picture> element

Making images scale in size is straightforward, but does not change the downloaded size of the image

```
img {  
    max-width: 100%;  
}
```

HTML5.1 defines the new <picture> element that lets the designer specify multiple elements. The browser determines which to use based on the viewport size.

```
<picture>  
  <source media="(min-width:960px)"  
    srcset="images/828-large.jpg">  
    Is this true?  
    if yes, then use this as the src for the <img>  
  
  <source media="(min-width:480px)"  
    srcset="images/828-medium.jpg">  
    Is this true?  
    if yes, then use this as the src for the <img>  
  
    
  </picture>  
    Otherwise use the src specified in the <img>
```



Supporting Material

- Code Repository for all chapters
<https://github.com/skanderturki/se371>
- For flex Layout understanding 15min quick video:
<https://www.youtube.com/watch?v=fYq5PXgSsbE>
- Grid min content, max content, fit content
<https://www.youtube.com/watch?v=DM244V9KvNs&t=29s>
- <https://css-tricks.com/auto-sizing-columns-css-grid-auto-fill-vs-auto-fit/>